
Dhago

Bridging the Credit Gap for Nepal's Informal Economy Using
Document-Income Consensus

Technical Documentation

Team: ZeroShot

Hackathon: GIBL AI/ML Hackathon 2026 – Track A

Repository: `autonomous-credit-and-lending-orchestrator`

Target: Nepal's Informal Economy

A five-agent autonomous credit scoring system addressing financial inclusion for Nepal's 63% unbanked population.

July 4, 2026

Contents

1	README	3
1.1	Overview	3
1.2	Setup & Installation	3
1.2.1	Prerequisites	3
1.2.2	Step 1 – Clone the Repository	4
1.2.3	Step 2 – Backend Setup	4
1.2.4	Step 3 – Pull the Ollama Model	4
1.2.5	Step 4 – Environment Variables	4
1.2.6	Step 5 – Database Setup	4
1.2.7	Step 6 – Frontend Setup	4
1.2.8	Step 7 – Load Dataset	4
1.3	Quick Start	5
1.3.1	Submit a Test Application	5
2	Architecture & System Design	5
2.1	System Design	5
2.1.1	Client Layer	6
2.1.2	Application Layer	6
2.1.3	Data Layer	7
2.2	Data Flow	7
2.3	Design Decisions	8
2.3.1	1. Document-income consensus over single-source trust	9
2.3.2	2. Compliance Agent as a structurally separate veto layer	9
2.3.3	3. XGBoost over deep learning	9
2.3.4	4. Ollama for all LLM inference (self-hosted)	9
2.3.5	5. Hierarchical income evaluation	9
2.3.6	6. Graceful degradation over hard failure	9
2.3.7	7. Sparse data model treated as expected, not as missing	9
2.4	Diagram with Flow	10
3	API & Reference	10
3.1	Class and Function Definitions	10
3.1.1	SignupRequest	11
3.1.2	LoginRequest	11
3.1.3	RefreshTokenRequest	11
3.1.4	LoanApplicationRequest (multipart/form-data)	12
3.1.5	AdminReviewRequest	12
3.2	Endpoints and Requests	12
3.2.1	Health	12
3.2.2	Auth	12
3.2.3	Loan	14
3.2.4	Admin	16
3.3	Error Handling	17
4	Guides, Tutorials, and Examples	18
4.1	Step-by-Step Tutorials	18
4.1.1	Tutorial 1 – A Farmer Applies for an Agricultural Loan	18
4.1.2	Tutorial 2 – Application Referred to Manual Review (Thin-File Fallback)	20
4.1.3	Tutorial 3 – Application Rejected by Compliance Agent	21
4.2	Usage Examples	21

4.2.1	Example 1 – Full Application via curl	21
4.2.2	Example 2 – Admin Reviewing Manual Review Queue	22
4.2.3	Example 3 – Refresh an Expired Token	22
5	Inline Code Comments and Docstrings	23
5.1	Docstrings	23
5.1.1	Document Agent	23
5.1.2	Income Agent	23
5.1.3	Scoring Agent	24
5.1.4	Compliance Agent	25

1 README

1.1 Overview

Project Identity

Project Name: Dhago – Autonomous Credit & Lending Orchestrator
Team: ZeroShot
Hackathon: GIBL AI/ML Hackathon 2026 – Track A
Repository: GitHub Repository

Dhago is a **five-agent autonomous credit scoring system** built for Nepal's informal economy. It addresses the core problem that **63%** of Nepal's economically active population – smallholder farmers, daily wage workers, remittance-dependent households – cannot access formal bank credit because they lack conventional employment documentation.

The system works on a principle called **document-income consensus**: it cross-validates declared income from government-issued documents against behavioral signals derived from digital transaction history (eSewa, Khalti, bank statements). When both evidence streams agree, that consistency is treated as a creditworthiness signal. When they diverge significantly, it is flagged as a fraud signal via the **Income Mismatch Ratio (IMR)**.

The pipeline runs as **five sequential agents** orchestrated via LangGraph:

Agent	Responsibility
Document Agent	OCR extraction from Nagrikta, Lalpurja, Tax Clearance docs
Income Agent	Aggregates 6 income source types, computes EMI · IMR · DTI · LTI
Scoring Agent	XGBoost model producing credit score 300–850 with SHAP explainability
Compliance Agent	Enforces NRB Unified Directives 2080 with unconditional veto authority
Decision Agent	Produces final decision with plain-language explanation in Nepali and English

Table 1: Five-Agent Pipeline

Guarantee

Every decision produces a full audit trail, SHAP-based explanation, and is **NRB Directive 2080 compliant**.

1.2 Setup & Installation

1.2.1 Prerequisites

- Python 3.11+
- Node.js 18+
- Ollama installed and running locally
- Git
- Docker

1.2.2 Step 1 – Clone the Repository

```
1 git clone https://github.com/sagarnewpane/autonomous-credit-and-lending-orchestrator
2 cd autonomous-credit-and-lending-orchestrator
```

1.2.3 Step 2 – Backend Setup

```
1 cd backend
2 python -m venv venv
3 source venv/bin/activate      # Windows: venv\Scripts\activate
4 pip install -r requirements.txt
```

1.2.4 Step 3 – Pull the Ollama Model

```
1 ollama pull llama3.3
```

This model is used by the **Income Agent** for transaction classification and by the **Decision Agent** for generating plain-language explanations.

1.2.5 Step 4 – Environment Variables

Create a `.env` file in the `backend/` directory:

```
1 DATABASE_URL=postgresql://user:password@localhost:5432/dhago
2 OLLAMA_BASE_URL=http://localhost:11434
3 MODEL_NAME=llama3.3
4 SECRET_KEY=your_secret_key_here
```

1.2.6 Step 5 – Database Setup

```
1 python scripts/init_db.py
2 docker is used for hosting, postgresql and pgadmin for the database
```

1.2.7 Step 6 – Frontend Setup

```
1 cd ../frontend
2 npm install
```

1.2.8 Step 7 – Load Dataset

Place the hackathon dataset files in `backend/data/`:

```
1 app/
2 |-- structured/
3 |   |-- applicant_profiles.csv
4 |   |-- loan_applications.csv
5 |   |-- mobile_money_transactions.parquet
6 |   |-- remittance_records.parquet
7 |   |-- utility_payments.parquet
8 |   |-- cooperative_members.csv
9 |   |-- cooperative_sales.csv
10 |   '-- document_registry.csv
11 '-- ground_truth/
```

1.3 Quick Start

Run the Backend

```
1 source venv/bin/activate
2 uvicorn app.main:app --reload --port 8000
```

Run the Frontend – Separate Terminal

```
1 cd frontend
2 npm run dev
```

Resource	URL
Application	http://localhost:5173
API	http://localhost:8000
API Documentation	http://localhost:8000/docs
Ollama	http://localhost:8081

Table 2: Service Endpoints

1.3.1 Submit a Test Application

```
1 curl -X POST http://localhost:8000/api/v1/assess \
2   -H "Content-Type: application/json" \
3   -d '{
4     "applicant_id": "AP-050234",
5     "loan_purpose": "agricultural_input",
6     "requested_amount_nrs": 150000,
7     "requested_tenure_months": 24
8   },'
```

2 Architecture & System Design

2.1 System Design

The system follows a **three-layer architecture** as required by the hackathon guidelines.

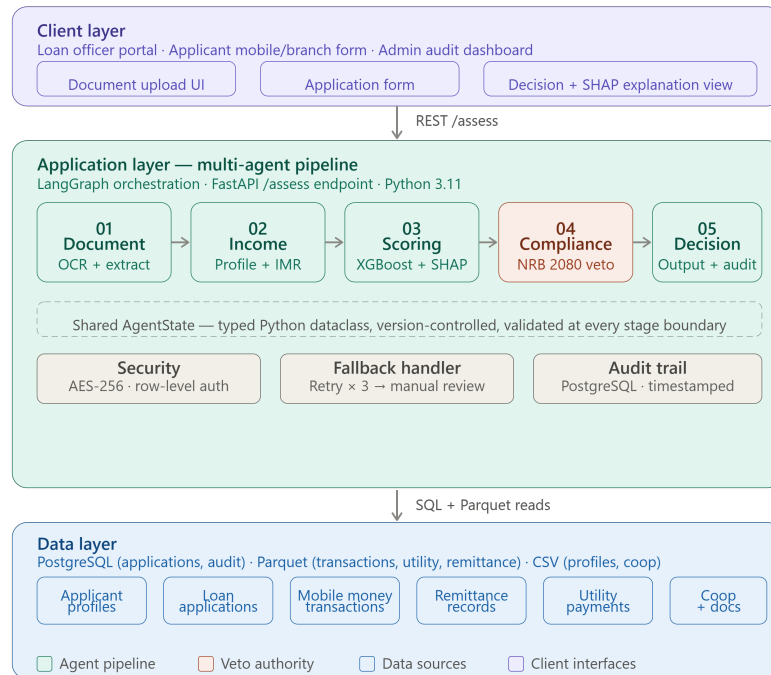


Figure 1: System Design

2.1.1 Client Layer

Built with **SvelteKit**. Provides three interfaces:

- **Document upload UI** — applicant submits citizenship certificate, tax clearance, Lalpurja scans
- **Application form** — loan purpose, requested amount, tenure
- **Decision view** — displays final decision, approved amount, interest rate, and SHAP-based explanation in plain Nepali and English

2.1.2 Application Layer

Built with **FastAPI** and orchestrated via **LangGraph**. Exposes a single primary endpoint (POST /api/v1/assess) that triggers the five-agent pipeline. All agent communication passes through a shared typed `AgentState` dataclass that is validated at every stage boundary.

The five agents run sequentially:

Agent	Technology	Responsibility
Document Agent	Ollama VLM + ChandraOCR	OCR, field extraction, doc_trust_score
Income Agent	Ollama Gemmae4B + Pandas	Income profiling, IMR, EMI, DTI, LTI
Scoring Agent	XGBoost + SHAP	Credit score 300–850, top 3 risk drivers
Compliance Agent	Deterministic rule engine	NRB 2080 enforcement, unconditional veto
Decision Agent	Ollama Gemmae4B	Final output, plain-language explanation, audit trail

Table 3: Agent Technology Stack

Cross-cutting Concerns

- **Access Control** – JWT-based authentication on all endpoints
- **Session Management** – stateless JWT tokens, expiry enforced server-side
- **Encryption** – AES-256 for all PII fields at rest
- **Hashed Passwords** – bcrypt hashing for all user accounts
- **Input Validations** – Pydantic models enforce type, range, format
- **Parameterized Queries** – SQLAlchemy ORM throughout, no raw SQL
- **Security Headers** – CORS, X-Content-Type-Options, X-Frame-Options
- **Error Handling** – structured JSON with error codes, no stack traces
- **Traceability** – every pipeline run logged to PostgreSQL audit table

2.1.3 Data Layer

Source	Format	Used by
applicant_profiles	CSV	All agents
loan_applications	CSV	Compliance Agent, Decision Agent
mobile_money_transactions	Parquet	Income Agent
remittance_records	Parquet	Income Agent
utility_payments	Parquet	Income Agent
cooperative_members / sales	CSV	Income Agent
document_registry	CSV	Document Agent
audit_trail	PostgreSQL	Decision Agent writes, all agents read

Table 4: Data Sources

2.2 Data Flow

The below figure is detailed data flow of the project.

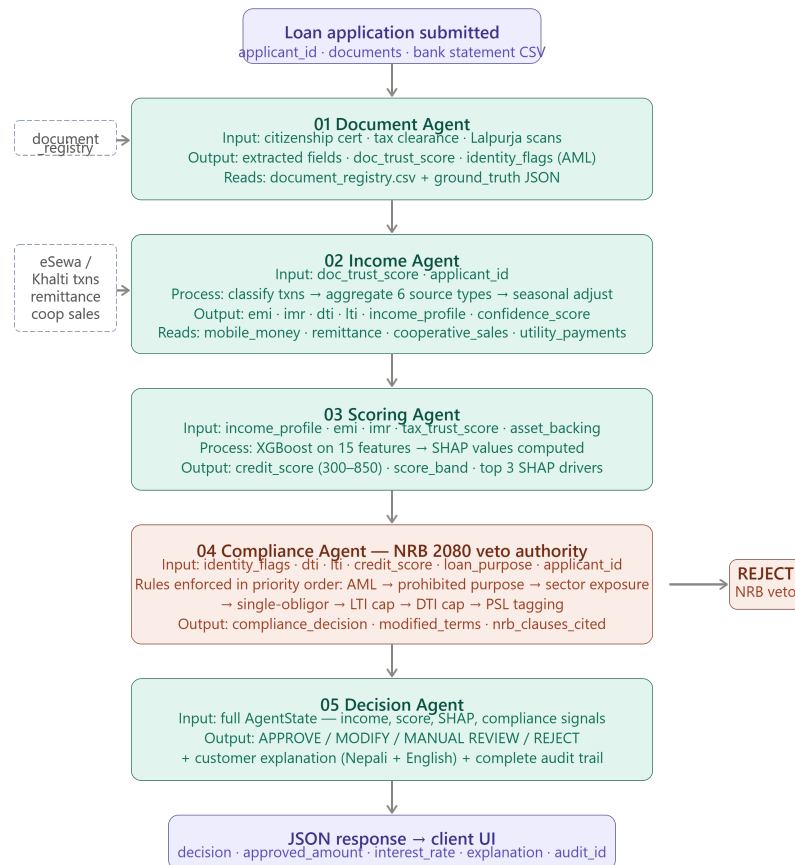


Figure 2: Data Flow

2.3 Design Decisions

Key design decisions

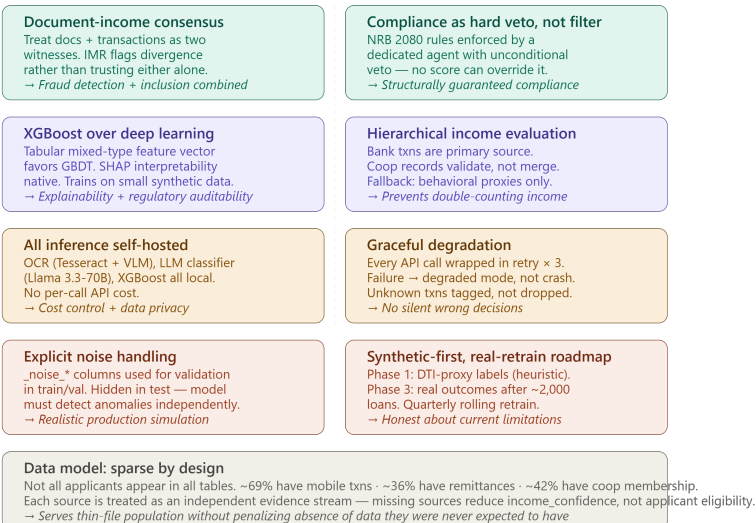


Figure 3: Design Decisions

2.3.1 1. Document-income consensus over single-source trust

Rather than accepting submitted documents at face value or relying solely on transaction data, the system cross-validates both. The **Income Mismatch Ratio** ($\text{IMR} = \text{declared income} / \text{observed income}$) operationalizes this.

- IMR close to **1.0** → signals credibility
- $\text{IMR} > 3.0$ → flags potential document inflation
- $\text{IMR} < 0.3$ → flags undisclosed income

This single design decision simultaneously improves **fraud detection** and serves **legitimate informal earners**.

2.3.2 2. Compliance Agent as a structurally separate veto layer

NRB rules are not embedded in the scoring model or as post-hoc filters. They are enforced by a **dedicated deterministic rule engine** that runs after scoring with **unconditional veto authority**. This guarantees regulatory compliance is a *structural property* of the system, not a behavioral one.

2.3.3 3. XGBoost over deep learning

The feature vector is tabular with mixed types (continuous ratios, binary flags, categorical income source labels). XGBoost:

- Outperforms deep learning on this structure
- Produces stable feature importances
- Supports SHAP natively
- Trains reliably on small synthetic datasets

2.3.4 4. Ollama for all LLM inference (self-hosted)

All LLM calls run through Ollama locally. This eliminates per-call API costs, keeps all PII on-premise, and removes external API dependency during the hackathon demo.

2.3.5 5. Hierarchical income evaluation

Bank transaction data is the primary income source. Cooperative records and utility payments are used as validators and confidence boosters – not merged into the income total. This prevents double-counting. When bank data is insufficient, the system activates a fallback pathway using behavioral proxies rather than rejecting the applicant outright.

2.3.6 6. Graceful degradation over hard failure

Every external call (Ollama, database queries, Parquet reads) is wrapped in a retry handler with exponential backoff (max 3 attempts). On exhausted retries, the affected agent marks its output fields as unavailable and routes to manual review with a system note. **Infrastructure failures never produce silent wrong decisions.**

2.3.7 7. Sparse data model treated as expected, not as missing

Not all applicants appear in all tables.

- ~31% have no mobile wallet data

- ~58% have no remittance records
- ~59% have no cooperative membership

Missing sources reduce `income_confidence` but do **not** reduce eligibility. This design choice directly serves the thin-file population the system is built for.

2.4 Diagram with Flow

The below figure is detailed diagram with flow of the project.

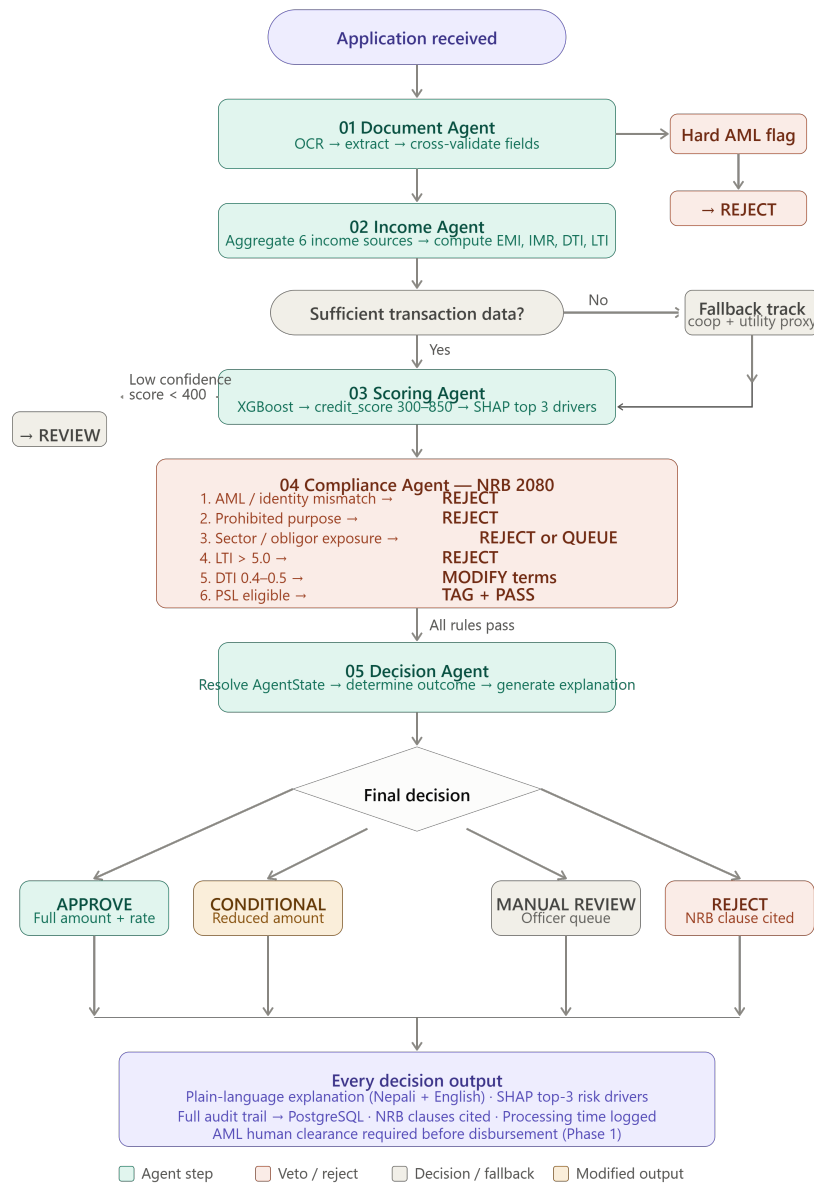


Figure 4: Diagram with Flow

3 API & Reference

3.1 Class and Function Definitions

API Base Configuration

Base URL: http://localhost:8000
API Prefix: /api/v1
Authentication: Bearer JWT token (required on all endpoints except signup/login)

Authentication Flow:

1. User calls POST /auth/signup or POST /auth/login
2. Server returns access_token and refresh_token
3. All subsequent requests include Authorization: Bearer <access_token> header
4. When access token expires, call POST /auth/refresh with the refresh token

Request Content Types:

- Auth endpoints → application/json
- Loan apply + document upload → multipart/form-data
- Admin review → application/json

3.1.1 SignupRequest

Field	Type	Required	Description
email	string	Yes	Valid email address
password	string	Yes	Minimum 8 characters

3.1.2 LoginRequest

Field	Type	Required	Description
email	string	Yes	Registered email
password	string	Yes	Account password

3.1.3 RefreshTokenRequest

Field	Type	Required	Description
refresh_token	string	Yes	Valid refresh token from prior login

3.1.4 LoanApplicationRequest (multipart/form-data)

Field	Type	Req.	Description
applicant_name	string	Yes	Full name of applicant
user_id	string	Yes	Authenticated user ID
loan_amount	integer	Yes	Requested amount in NRs
loan_purpose	string	Yes	e.g. agricultural_input, home_repair
tenure_months	integer	Yes	One of: 6, 12, 18, 24, 36
monthly_income	integer	Yes	Self-declared monthly income in NRs
monthly_debt	integer	Yes	Existing monthly debt obligations in NRs
collateral_type	float	Yes	1.0 = land (Lalpurja), 0.0 = none
cooperative_member	boolean	Yes	true or false

Table 5: Loan Application Fields

3.1.5 AdminReviewRequest

Field	Type	Req.	Description
admin_status	string	Yes	approved, rejected, or referred
admin_notes	string	Yes	Officer's review remarks
reviewed_by	string	Yes	Admin email performing the review

3.2 Endpoints and Requests

3.2.1 Health

```
1 GET /api/v1/loan/
```

Health check. Confirms the API is running.

Auth required: No **Request body:** None

Response 200 OK:

```
1 {
2   "status": "ok"
3 }
```

3.2.2 Auth

POST /api/v1/auth/signup – Register a new user account.

Request body:

```
1 {
2   "email": "testuser@example.com",
3   "password": "securepassword123"
4 }
```

Response 201 Created:

```
1 {
2   "access_token": "<jwt_token>",
3   "refresh_token": "<refresh_token>",
4   "user": {
5     "id": "user_id",
6     "email": "testuser@example.com"
7   }
8 }
```

POST /api/v1/auth/login – Authenticate an existing user.

Request body:

```
1 {
2   "email": "testuser@example.com",
3   "password": "securepassword123"
4 }
```

Response 200 OK:

```
1 {
2   "access_token": "<jwt_token>",
3   "refresh_token": "<refresh_token>",
4   "user": {
5     "id": "user_id",
6     "email": "testuser@example.com"
7   }
8 }
```

POST /api/v1/auth/refresh – Exchange refresh token for new access token.

Request body:

```
1 {
2   "refresh_token": "<refresh_token>"
3 }
```

Response 200 OK:

```
1 {
2   "access_token": "<new_jwt_token>",
3   "refresh_token": "<new_refresh_token>"
4 }
```

GET /api/v1/auth/me – Get the currently authenticated user's profile.

Response 200 OK:

```
1 {
2   "id": "user_id",
3   "email": "testuser@example.com"
4 }
```

POST /api/v1/auth/logout – Invalidate current session token.

Response 200 OK:

```
1 {
2   "message": "Logged out successfully"
3 }
```

POST /api/v1/auth/login (Admin) – Same structure as regular login, returns admin-level permissions.

Response 200 OK:

```
1 {
2   "access_token": "<admin_jwt_token>",
3   "refresh_token": "<admin_refresh_token>",
4   "user": {
5     "id": "admin_id",
6     "email": "admin@example.com",
7     "role": "admin"
8   }
9 }
```

3.2.3 Loan

POST /api/v1/loan/apply – Submit a new loan application. Triggers the full five-agent pipeline: Document → Income → Scoring → Compliance → Decision.

Request fields (multipart/form-data):

```
1 applicant_name      : "John Doe"
2 user_id             : "<testUserId>"
3 loan_amount         : 500000
4 loan_purpose          : "Home Renovation"
5 tenure_months      : 24
6 monthly_income      : 80000
7 monthly_debt        : 15000
8 collateral_type     : 1.0
9 cooperative_member  : true
```

Response 200/201:

```
1 {
2   "application_id": "LA-2081-050234",
3   "status": "processing",
4   "message": "Application submitted. Pipeline initiated."
5 }
```

GET /api/v1/loan/decision/{application_id} – Retrieve the final decision.

Response 200 OK:

```
1 {
2   "application_id": "LA-2081-050234",
3   "final_decision": "approve",
4   "approved_amount_nrs": 500000,
5   "interest_rate_pct": 11.5,
6   "interest_tier": "premium",
7   "credit_score": 712,
```

```
8   "score_band": "very_good",
9   "compliance_status": "pass",
10  "compliance_flags": [],
11  "income_agent_monthly_est": 78000,
12  "income_confidence": 0.81,
13  "processing_time_seconds": 47
14 }
```

GET /api/v1/loan/explain/{application_id} – Retrieve SHAP-based plain-language explanation.

Response 200 OK:

```
1 {
2   "application_id": "LA-2081-050234",
3   "explanation_en": "Your application was approved. The top factors were:
4   consistent eSewa transaction history over 6 months, low existing debt
5   obligations relative to estimated income, and valid land collateral (
6   Lalpurja) submitted.",
7   "explanation_np": "...",
8   "shap_drivers": [
9     { "feature": "cumulative_on_time_rate", "contribution": 0.18, "
10    direction": "positive" },
11    { "feature": "repayment_pressure_ratio", "contribution": -0.12, "
12    direction": "positive" },
13    { "feature": "has_lalpurja", "contribution": 0.09, "direction": "
14    positive" }
15  ]
16 }
```

PUT /api/v1/loan/docs/{application_id} – Re-upload documents.

Response 200 OK:

```
1 {
2   "application_id": "LA-2081-050234",
3   "message": "Documents updated. Re-verification initiated."
4 }
```

GET /api/v1/loan/user/{user_id} – Retrieve loan application history for a user.

Response 200 OK:

```
1 {
2   "user_id": "user_id",
3   "applications": [
4     {
5       "application_id": "LA-2081-050234",
6       "loan_purpose": "Home Renovation",
7       "requested_amount_nrs": 500000,
8       "final_decision": "approve",
9       "application_date_ad": "2024-04-15"
10    }
11  ]
12 }
```


GET /api/v1/loan/compliance/references/{application_id} – Retrieve NRB Unified Directive 2080 clauses evaluated.

Response 200 OK:

```
1 {
2   "application_id": "LA-2081-050234",
3   "compliance_status": "pass",
4   "compliance_flags": [],
5   "nrb_clauses_evaluated": [
6     { "clause": "NRB-AML-004", "result": "pass" },
7     { "clause": "NRB-LTI-002", "result": "pass", "value": 6.41 },
8     { "clause": "NRB-DTI-clause-5.2", "result": "pass", "value": 0.38 },
9     { "clause": "NRB-COL-005", "result": "pass" }
10  ]
11 }
```

3.2.4 Admin

GET /api/v1/admin/loans – Retrieve all loan applications. Admin access required.

Response 200 OK:

```
1 {
2   "total": 1240,
3   "applications": [
4     {
5       "application_id": "LA-2081-050234",
6       "applicant_name": "John Doe",
7       "final_decision": "approve",
8       "credit_score": 712,
9       "application_date_ad": "2024-04-15"
10    }
11  ]
12 }
```

PATCH /api/v1/admin/review/{application_id} – Submit a manual review decision.

Request body:

```
1 {
2   "admin_status": "approved",
3   "admin_notes": "Verified all documents. Applicant qualifies.",
4   "reviewed_by": "admin@example.com"
5 }
```

Response 200 OK:

```
1 {
2   "application_id": "LA-2081-050234",
3   "admin_status": "approved",
4   "reviewed_by": "admin@example.com",
5   "reviewed_at": "2024-04-15T14:30:00Z"
6 }
```

GET /api/v1/admin/audit/{application_id} – Retrieve full timestamped audit trail.

Response 200 OK:

```
1 {
2   "application_id": "LA-2081-050234",
3   "audit_trail": [
4     {
5       "agent": "DocumentAgent",
6       "timestamp": "2024-04-15T09:00:01Z",
7       "output": { "doc_trust_score": 0.91, "identity_flags": [] }
8     },
9     {
10      "agent": "IncomeAgent",
11      "timestamp": "2024-04-15T09:00:08Z",
12      "output": { "emi": 78000, "imr": 1.02, "dti": 0.38, "lti": 6.41 }
13    },
14    {
15      "agent": "ScoringAgent",
16      "timestamp": "2024-04-15T09:00:22Z",
17      "output": { "credit_score": 712, "score_band": "very_good" }
18    },
19    {
20      "agent": "ComplianceAgent",
21      "timestamp": "2024-04-15T09:00:31Z",
22      "output": { "compliance_status": "pass", "clauses_evaluated": 4 }
23    },
24    {
25      "agent": "DecisionAgent",
26      "timestamp": "2024-04-15T09:00:47Z",
27      "output": { "final_decision": "approve", "approved_amount_nrs":
500000 }
28    }
29  ]
30 }
```

3.3 Error Handling

All errors return a consistent JSON structure:

```
1 {
2   "error": {
3     "code": "ERROR_CODE",
4     "message": "Human-readable description",
5     "details": {}
6   }
7 }
```

HTTP	Error Code	Meaning
400	VALIDATION_ERROR	Request body failed Pydantic validation
400	INVALID_LOAN_PURPOSE	loan_purpose not in allowed enum
400	NEGATIVE_AMOUNT	loan_amount is zero or negative
401	UNAUTHORIZED	Missing or invalid Bearer token
401	TOKEN_EXPIRED	Access token expired – call /auth/refresh
403	FORBIDDEN	Valid token but insufficient permissions
404	APPLICATION_NOT_FOUND	application_id does not exist
404	USER_NOT_FOUND	user_id does not exist
409	DUPLICATE_APPLICATION	Same applicant has active application within 90 days (NRB-DUP-006)
422	PIPELINE_DEGRADED	One or more agents failed after retries – routed to manual review
500	INTERNAL_ERROR	Unexpected server error – logged to audit trail

Table 6: HTTP Error Codes

NRB Compliance Rejection CodesReturned in `compliance_flags` field:

Code	Rule	Meaning
NRB_AML_004	AML / identity mismatch	Hard reject – do not reapply without identity verification
NRB_LTI_002	Loan-to-Income > 5.0	Loan amount too high relative to estimated income
NRB_DTI_052	Debt-to-Income > 0.5	Total debt obligations exceed 50% of monthly income
NRB_SEC_003	Sector exposure breach	Bank's sector concentration limit reached
NRB_COL_005	Collateral required	Loans above NRs 5,00,000 require Lalpurja
LTI_EXCEEDED	LTI soft flag	Loan amount adjusted downward to comply

Table 7: NRB Compliance Codes

4 Guides, Tutorials, and Examples

4.1 Step-by-Step Tutorials

4.1.1 Tutorial 1 – A Farmer Applies for an Agricultural Loan

This tutorial walks through the complete user journey for **Ram Bahadur Thapa**, a paddy farmer from Kavrepalanchok who needs NRs 150,000 for agricultural inputs (seeds, fertilizer) before the Kharif planting season.

Step 1 – Create an Account. Ram visits the Dhago web application and registers.

```
1 POST /api/v1/auth/signup
```

```
1 {
2   "email": "ram.thapa@example.com",
3   "password": "securepass123"
4 }
```

The server returns an `access_token` and `refresh_token`. Ram's client stores both. All subsequent requests include `Authorization: Bearer <access_token>`.

Step 2 – Submit the Loan Application. Ram fills the application form. He uploads a photo of his *Nagriкта* (citizenship certificate) and his *Lalpurja* (land deed). He does not have a formal payslip – his income comes from paddy sales through Kavrepalanchok Krishak Sahakari Sanstha Ltd. and occasional eSewa wallet top-ups.

```
1 POST /api/v1/loan/apply (multipart/form-data)
2 applicant_name      : "Ram Bahadur Thapa"
3 user_id             : "usr_ram_001"
4 loan_amount         : 150000
5 loan_purpose          : "agricultural_input"
6 tenure_months      : 12
7 monthly_income      : 25000
8 monthly_debt        : 0
9 collateral_type     : 1.0
10 cooperative_member : true
```

The server responds immediately:

```
1 {
2   "application_id": "LA-2081-050234",
3   "status": "processing",
4   "message": "Application submitted. Pipeline initiated."
5 }
```

Step 3 – Pipeline Runs Automatically. Behind the scenes, LangGraph triggers all five agents sequentially:

1. **Document Agent** extracts Ram's citizenship number, father's name, and land area. `doc_trust_score = 0.87`. No identity mismatches found.
2. **Income Agent** finds 4 months of eSewa transaction history and 2 cooperative sale records (Paddy: 800kg at NRs 35/kg = NRs 28,000 last Kharif season). Seasonal adjustment applied. `emi = 23,500`, `imr = 0.94`, `dti = 0.0`, `income_confidence = 0.72`.
3. **Scoring Agent** builds the 15-feature vector. XGBoost produces `credit_score = 684`, `score_band = "very_good"`. SHAP top drivers: cooperative membership tenure (+), on-time utility payment rate (+), thin digital transaction history (–).
4. **Compliance Agent** runs all NRB 2080 rules:
 - $LTI = 150,000 / (23,500 \times 12) = 0.53 \rightarrow$ within NRB-LTI-002 (max 5.0) ✓
 - $DTI = 0 / 23,500 = 0.0$ ✓
 - Lalpurja present, loan below NRs 5,00,000 ✓
 - Agricultural loan purpose $\rightarrow$ PSL-eligible $\rightarrow$ tagged ✓
 - All rules pass $\rightarrow$ `compliance_status = "pass"`
5. **Decision Agent** resolves to **APPROVE**. Calls Ollama Llama 3.3 to generate explanation. Writes audit trail to PostgreSQL.

Step 4 – Retrieve the Decision

```
1 GET /api/v1/loan/decision/LA-2081-050234
```

```
1 {
2   "application_id": "LA-2081-050234",
3   "final_decision": "approve",
4   "approved_amount_nrs": 150000,
5   "interest_rate_pct": 12.5,
6   "interest_tier": "premium",
7   "credit_score": 684,
8   "score_band": "very_good",
9   "compliance_status": "pass",
10  "compliance_flags": [],
11  "income_agent_monthly_est": 23500,
12  "income_confidence": 0.72,
13  "processing_time_seconds": 43
14 }
```

Step 5 – Read the Explanation

```
1 GET /api/v1/loan/explain/LA-2081-050234
```

```
1 {
2   "application_id": "LA-2081-050234",
3   "explanation_en": "Your loan of NRs 1,50,000 has been approved at 12.5%
4   annual interest. The three strongest factors in your favour were: your
5   active cooperative membership and consistent sales history, your regular
6   utility bill payments with no arrears, and your land collateral (Lalpurja
7   ). Your limited eSewa transaction history slightly reduced your score,
8   which you can improve by using digital payments more regularly.",
9   "explanation_np": "...",
10  "shap_drivers": [
11    { "feature": "cooperative_membership_tenure", "contribution": 0.21, "
12    direction": "positive" },
13    { "feature": "cumulative_on_time_rate", "contribution": 0.17, "
14    direction": "positive" },
15    { "feature": "months_of_data", "contribution": -0.09, "direction": "
16    negative" }
17  ]
18 }
```

4.1.2 Tutorial 2 – Application Referred to Manual Review (Thin-File Fallback)

Sita Devi Tamang is a 58-year-old subsistence farmer in Karnali with no mobile wallet, no remittance history, and no cooperative membership. She applies for NRs 80,000 for livestock purchase.

What happens differently:

- **Document Agent** extracts her Nagrikta successfully. doc_trust_score = 0.79 (low DPI mobile camera scan).
- **Income Agent** finds zero eSewa/Khalti transactions and zero remittance records. Activates fallback pathway. Pulls utility payment history (NEA electricity, 6 months, cumulative_on_time_rate = 0.83). Estimates income from declared occupation (Farmer) and household size. income_confidence = 0.31 – below the 0.40 threshold.
- **Scoring Agent** produces credit_score = 388, score_band = "poor" due to near-zero behavioral data.
- Because credit_score < 400 AND income_confidence < 0.40, the pipeline routes to **MANUAL REVIEW** before Compliance Agent runs.

```
1 GET /api/v1/loan/decision/LA-2081-060891
```

```
1 {
2   "application_id": "LA-2081-060891",
3   "final_decision": "refer",
4   "approved_amount_nrs": null,
5   "credit_score": 388,
6   "score_band": "poor",
7   "income_confidence": 0.31,
8   "processing_time_seconds": 38,
9   "review_note": "Insufficient behavioral data. Routed to officer queue.
  Applicant may provide cooperative passbook or guarantor letter to
  strengthen application."
10 }
```

A loan officer sees this in their admin queue and can call:

```
1 PATCH /api/v1/admin/review/LA-2081-060891
```

```
1 {
2   "admin_status": "approved",
3   "admin_notes": "Officer verified cooperative passbook in-branch. Income
  consistent with declared occupation. Approving NRs 80,000 with guarantor
  .",
4   "reviewed_by": "officer.karnali@globalimebank.com"
5 }
```

4.1.3 Tutorial 3 – Application Rejected by Compliance Agent

A trader applies for NRs 9,00,000 for vehicle purchase with no collateral.

- **Scoring Agent** produces `credit_score = 720` – strong score.
- **Compliance Agent** evaluates NRB-COL-005: loans above NRs 5,00,000 require registered collateral. `collateral_type = "none"`. → **Hard veto** → **REJECT**.

Key Principle

The strong credit score does not matter. The compliance veto is **unconditional**.

```
1 GET /api/v1/loan/decision/LA-2081-071122
```

```
1 {
2   "application_id": "LA-2081-071122",
3   "final_decision": "reject",
4   "approved_amount_nrs": null,
5   "credit_score": 720,
6   "compliance_status": "flag",
7   "compliance_flags": ["NRB_COL_005"],
8   "explanation_en": "Your application was rejected because loans above NRs
  5,00,000 require registered land collateral (Lalpurja) under NRB Unified
  Directives 2080. Your credit score is strong. To reapply, either submit a
  Lalpurja or reduce the requested amount to NRs 5,00,000 or below."
9 }
```

4.2 Usage Examples

4.2.1 Example 1 – Full Application via curl

```
1 # Step 1: Login
2 curl -X POST http://localhost:8000/api/v1/auth/login \
3   -H "Content-Type: application/json" \
4   -d '{"email": "testuser@example.com", "password": "securepassword123"}'
5
6 # Step 2: Store the token (replace with actual token from response)
7 TOKEN="your_access_token_here"
8
9 # Step 3: Submit application
10 curl -X POST http://localhost:8000/api/v1/loan/apply \
11   -H "Authorization: Bearer $TOKEN" \
12   -F "applicant_name=Ram Bahadur Thapa" \
13   -F "user_id=usr_ram_001" \
14   -F "loan_amount=150000" \
15   -F "loan_purpose=agricultural_input" \
16   -F "tenure_months=12" \
17   -F "monthly_income=25000" \
18   -F "monthly_debt=0" \
19   -F "collateral_type=1.0" \
20   -F "cooperative_member=true"
21
22 # Step 4: Get decision (replace with actual application_id)
23 curl -X GET http://localhost:8000/api/v1/loan/decision/LA-2081-050234 \
24   -H "Authorization: Bearer $TOKEN"
25
26 # Step 5: Get explanation
27 curl -X GET http://localhost:8000/api/v1/loan/explain/LA-2081-050234 \
28   -H "Authorization: Bearer $TOKEN"
```

4.2.2 Example 2 – Admin Reviewing Manual Review Queue

```
1 ADMIN_TOKEN="your_admin_token_here"
2
3 # Get all loans pending review
4 curl -X GET http://localhost:8000/api/v1/admin/loans \
5   -H "Authorization: Bearer $ADMIN_TOKEN"
6
7 # Submit manual review decision
8 curl -X PATCH http://localhost:8000/api/v1/admin/review/LA-2081-060891 \
9   -H "Authorization: Bearer $ADMIN_TOKEN" \
10  -H "Content-Type: application/json" \
11  -d '{
12    "admin_status": "approved",
13    "admin_notes": "Verified cooperative passbook in-branch.",
14    "reviewed_by": "admin@globalimebank.com"
15  }'
16
17 # Pull full audit trail
18 curl -X GET http://localhost:8000/api/v1/admin/audit/LA-2081-060891 \
19   -H "Authorization: Bearer $ADMIN_TOKEN"
```

4.2.3 Example 3 – Refresh an Expired Token

```
1 curl -X POST http://localhost:8000/api/v1/auth/refresh \
2   -H "Content-Type: application/json" \
3   -d '{"refresh_token": "your_refresh_token_here"}'
```

5 Inline Code Comments and Docstrings

5.1 Docstrings

All agents, services, and utility functions follow **Google-style Python docstrings**. Below are representative examples for each agent.

5.1.1 Document Agent

```

1 class DocumentAgent:
2     """
3     Extracts and validates structured financial data from uploaded
4     government-issued documents using OCR and vision-language models.
5
6     Handles three document classes:
7     - Nagrikta (citizenship certificate)
8     - Lalpurja (land ownership certificate)
9     - Tax clearance certificate (Inland Revenue Department)
10
11     Uses Ollama VLM as primary extractor for Devanagari script fields.
12     Falls back to Tesseract 5.0 (nep+eng) for Latin-script fields.
13     Cross-validates citizenship numbers across submitted documents and
14     raises AML flags on hard identity mismatches.
15
16     Attributes:
17         ollama_client: Ollama client instance for VLM inference.
18         tesseract_config: Tesseract language config string.
19         trust_threshold: Minimum doc_trust_score to pass without flag.
20
21     Updates AgentState fields:
22         documents_parsed (dict): Extracted field dictionaries per document.
23         doc_trust_score (float): Composite trust score 0.0 to 1.0.
24         identity_flags (list[str]): AML or mismatch flags raised.
25     """
26
27     def process(self, state: AgentState) -> AgentState:
28         """
29         Run the full document extraction pipeline on uploaded files.
30
31         Args:
32             state (AgentState): Shared pipeline state. Must contain
33                 raw_documents (list[bytes]) populated by the API layer.
34
35         Returns:
36             AgentState: Updated state with documents_parsed,
37                 doc_trust_score, and identity_flags populated.
38
39         Raises:
40             DocumentExtractionError: If all documents fail extraction
41                 after three retry attempts.
42         """

```

5.1.2 Income Agent

```

1 class IncomeAgent:
2     """
3     Constructs a behavioral income profile from alternative data sources.
4
5     Aggregates signals from mobile money transactions (eSewa/Khalti),
6     remittance records, cooperative sales, and utility payment history.

```



```

7   Categorizes transaction inflows into six source types and computes
8   income stability, regularity, and cross-validation features.
9
10  Activates a fallback pathway for applicants with insufficient
11  transaction history, using cooperative passbook records, utility
12  payment regularity, and agricultural harvest cycle proxies as
13  behavioral income signals.
14
15  Updates AgentState fields:
16      income_profile (dict): Per-source income metrics.
17      emi (float): Effective Monthly Income in NRs.
18      imr (float): Income Mismatch Ratio (declared / observed).
19      dti (float): Debt-to-Income Ratio.
20      lti (float): Loan-to-Income Ratio.
21      income_confidence (float): Confidence score 0.0 to 1.0.
22  """
23
24  def compute_imr(
25      self,
26      declared_monthly_income: float,
27      observed_monthly_income: float
28  ) -> float:
29      """
30      Compute the Income Mismatch Ratio between declared and observed
31      income.
32
33      IMR = declared_monthly_income / observed_monthly_income
34
35      An IMR close to 1.0 indicates consistency between document-declared
36      and behaviorally-observed income. IMR > 3.0 flags potential document
37      inflation. IMR < 0.3 flags potential undisclosed income sources.
38
39      Args:
40          declared_monthly_income (float): Monthly income from tax
41          clearance
42          certificate, in NRs.
43          observed_monthly_income (float): Monthly income derived from
44          aggregated transaction inflows, in NRs.
45
46      Returns:
47          float: IMR value. Returns 0.0 if observed income is zero to
48          prevent division by zero (handled as thin-file case upstream)
49      """

```

5.1.3 Scoring Agent

```

1  class ScoringAgent:
2      """
3      Generates a credit score and SHAP-based explanation from the income
4      profile produced by the Income Agent.
5
6      Uses a gradient-boosted XGBoost classifier trained on a synthetic
7      dataset calibrated to Nepal's informal sector income distribution.
8      Produces a continuous score on a 300-850 scale and computes per-
9      application SHAP values for regulatory auditability.
10
11     Note:
12         The model is trained on synthetic data with DTI-proxy surrogate
13         labels. Until ~2,000 real loan outcome labels are accumulated,
14         outputs should be treated as a compliance-weighted heuristic,
15         not an empirical default probability.

```

```

16
17     Updates AgentState fields:
18         credit_score (int): Score 300 to 850.
19         score_band (str): poor / fair / good / very_good / excellent.
20         shap_drivers (list[dict]): Top 3 features by absolute SHAP value.
21         scoring_confidence (float): Derived from data completeness.
22     """
23
24     def build_feature_vector(self, state: AgentState) -> pd.DataFrame:
25         """
26         Construct the 15-feature input vector for XGBoost inference.
27
28         Features are drawn from the AgentState income_profile and
29         document fields. DTI and LTI are intentionally excluded from
30         the feature vector to prevent label leakage -- they are computed
31         separately and passed directly to the Compliance Agent.
32
33         Args:
34             state (AgentState): Pipeline state with income_profile
35                               and documents_parsed populated.
36
37         Returns:
38             pd.DataFrame: Single-row DataFrame with exactly 15 columns
39                           matching the model's training feature schema.
40         """

```

5.1.4 Compliance Agent

```

1 class ComplianceAgent:
2     """
3     Deterministic rule engine enforcing NRB Unified Directives 2080.
4
5     Evaluates each application against all applicable NRB rules in strict
6     priority order. Holds unconditional veto authority over all upstream
7     agent recommendations. No credit score, however favorable, can override
8     a compliance veto.
9
10    Rules are evaluated in the following order:
11        1. AML / hard identity mismatch (NRB-AML-004)
12        2. Prohibited loan purpose (NRB clause 4.2)
13        3. Sector / single-obligor exposure breach (NRB-SEC-003)
14        4. LTI > 5.0 (NRB-LTI-002)
15        5. DTI > 0.5 (NRB-DTI clause 5.2)
16        6. DTI 0.4-0.5 (modify terms)
17        7. PSL eligibility tagging
18
19    Every enforcement decision is logged with the specific directive
20    clause cited, calculated ratio, original and modified terms,
21    and a UTC timestamp.
22
23    Updates AgentState fields:
24        compliance_decision (str): pass / flag / veto.
25        compliance_flags (list[str]): NRB rule codes triggered.
26        modified_terms (dict): Adjusted amount/tenure if terms modified.
27    """

```